Assignment 2: K-Nearest-Neighbor Classifier

Note: Review the following sections for study purposes. Submit only the bug-free code for Steps 10 and 11 in Section 4 with either '.ipynb' format or '.txt' format to A2L by the due date (Due on Nov 4, 2026, 11:59 PM).

Section 1 – Measure of Distance

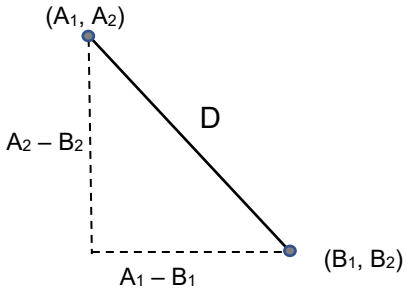
K-Nearest-Neighbor algorithm is often used for classification tasks. It assumes data points that belong to the same class would be close to each other. Before we are getting into how the algorithm works, let us first explore two common ways to define the distance between two data points.
The two types of distance we will cover are:

- **Euclidean Distance**
- **Manhattan Distance**

We would use Python lists to represent data points. For example, point (2026, 10) might be represented as `point_1 = [2026, 10]`. If we are dealing with data points that have more than two features, we can append more elements into the list, such as `point_2 = [2026, 10, 15, 1995, 12, 12]`.

Euclidean Distance might be the most commonly used measure of distance. You ought to be familiar with this distance. It is also called $l2$ norm. The distance formula is shown below:

$$Euclidean\ Distance(D) = \sqrt{(pointA_1 - pointB_1)^2 + (pointA_2 - pointB_2)^2 + \cdots + (pointA_n - pointB_n)^2}$$



$$D = \sqrt{(A_1 - B_1)^2 + (A_2 - B_2)^2}$$

To calculate Euclidean Distance between two points, we first subtract each corresponding feature between two points and then take the square root of the sum of the squared difference.

1. Let us write a function called euclidean to implement the calculation of Euclidean Distance. The function shall take two lists as arguments.

```
def euclidean(point_1, point_2):
    distance = 0
    for i in range(len(point_1)):
        distance += (point_1[i] - point_2[i]) ** 2
    return distance ** 0.5
```

2. Write another function to generate random 2-D points.

```
from random import seed
from random import randrange

seed(7)
def random_list(dimension):
    point = []
    for i in range(dimension):
        point.append(randrange(0, 51, 2))
    return point

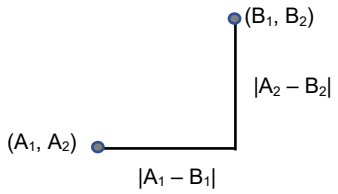
pt_1 = random_list(2)
pt_2 = random_list(2)
```

3. Pass pt_1 and pt_2 into the euclidean function to get the distance between these two points.

```
print(euclidean(pt_1, pt_2))
```

Manhattan Distance measures the distance between two points by computing the sum of absolutes. Imagine you are walking in city blocks, and thus, the route would only be orthogonal. It is also called $l1$ norm. The distance formula is shown below:

$$Manhattan\ Distance(D) = |pointA_1 - pointB_1| + |pointA_2 - pointB_2| + \cdots + |pointA_n - pointB_n|$$



$$D = |A_1 - B_1| + |A_2 - B_2|$$

1. Let us write a function called `manhattan` to implement the calculation of Manhattan Distance. The function shall take two lists as arguments.

```
def manhattan(point_1, point_2):
    distance = 0
    for i in range(len(point_2)):
        distance += abs(point_1[i] - point_2[i])
    return distance
```

2. Pass `pt_1` and `pt_2` into the `manhattan` function to get the distance between these two points.

```
print(manhattan(pt_1, pt_2))
```

Now let us move on to the next section, where we would briefly explore how KNN works.

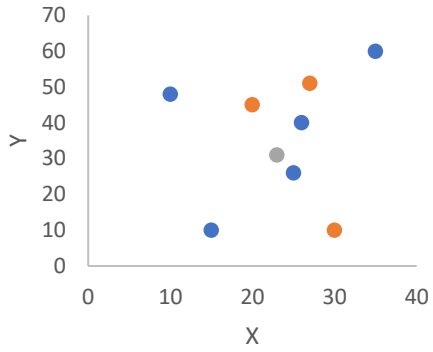
Section 2 – Walkthrough of the concept

As mentioned in Section 1, K-Nearest Neighbors classifier assumes that data points with similar features would be classified into similar categories. Let us look at the table below.

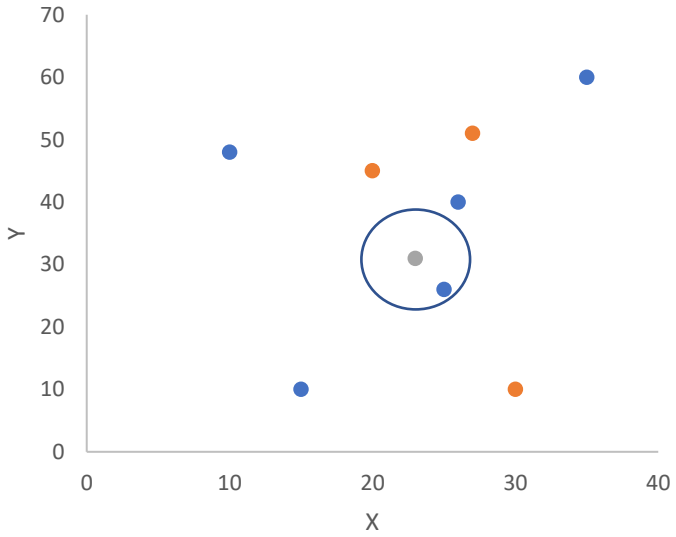
Table 1 - KNN range circle illustration

Blue points		Orange points		Gray point (unknown)	
x	y	x	y	x	y
15	10	20	45	23	31
26	40	27	51		
25	27	30	10		
35	60				
10	48				

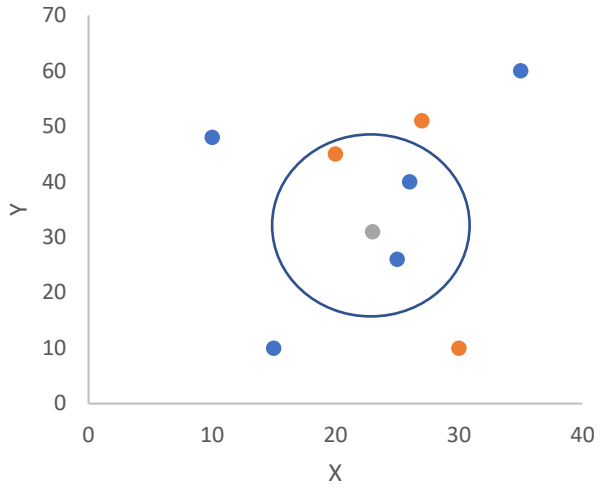
The objective of dealing with these data points in the table is to categorize them into either blue or orange class. If the point appears to be gray, the class of the point has not been decided yet. Those data points are visualized in this scatter plot:



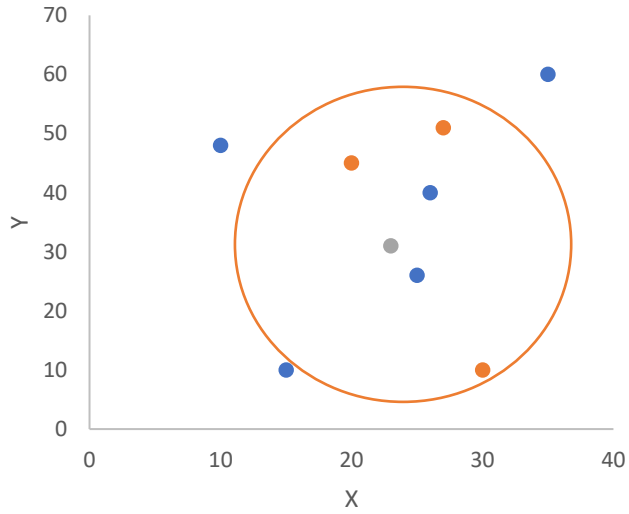
In order to classify the gray point using KNN, let us take a look at the expanding circle around the gray point. When there is only one point inside the circle(excluding the gray point), there is one ‘neighbor,’ so k=1. There is one blue point inside the circle so that the unknown point would be classified as blue.



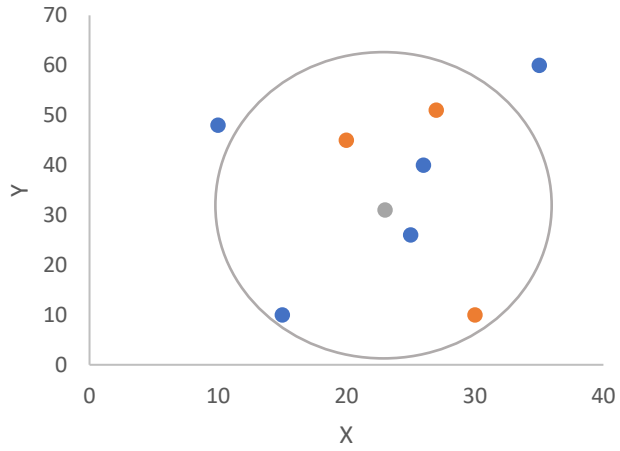
When k = 3, there are two blue points and one orange point inside the circle, so the unknown point would be classified as blue.



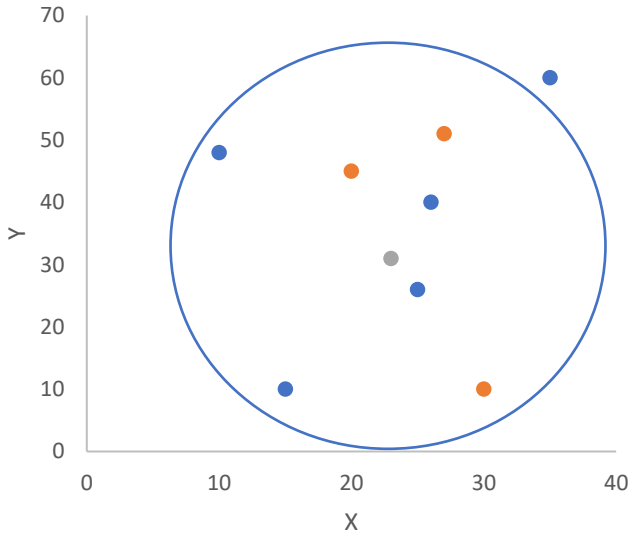
When k = 5, there are 2 blue points and 3 orange points inside the circle, so the unknown point would be classified as orange.



Although the number of k would often be odd numbers to avoid a potential tie situation, you may also select an even number of neighbors. When there is a tie, there are various ways to break the tie. For example, the sequence of the points being included in the circle might be taken into consideration. So if the blue point has been identified first, and the overall number of blue points exceeds the number of range points, then the gray point would be classified as blue.



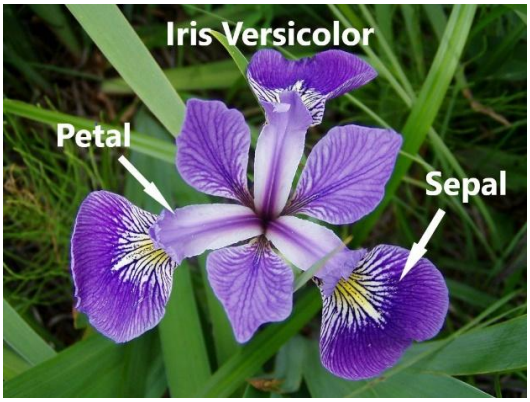
When k = 7, there are 4 blue points and 3 orange points inside the circle, so the unknown point would be classified as blue.



That would be the core idea of this algorithm. You can take an unknown point and classify it by using its nearest neighbors. We can visualize the points and then figure out which points are close to each other, but for programs, we need to define how to calculate the distance. We have introduced two types of distance in the first section. Generally, we would use Euclidean distance to calculate the distance between data points when using K-nearest-Neighbors.

Section 3 – Explore the iris dataset

We are going to use the iris dataset to illustrate the K-nearest-neighbors algorithm. This dataset contains 150 iris flowers of three different species: Iris setosa, Iris versicolor, and Iris virginica. The features are sepal length, sepal width, petal length, and petal width of the iris flowers.







1. Load the iris dataset from Sklearn's built-in datasets. Note that the iris variable is a dictionary that contains keys and the corresponding values. So we list the key names of the dictionary.

```
from sklearn import datasets

iris = datasets.load_iris()
list(iris.keys())
```

2. Check the characteristics of the dataset by accessing the 'DESCR' key.

```
print(iris['DESCR'])
```

3. Print the feature names of this dataset. Assign the features from iris['data'] to X. Assign the label from iris['target'] to y. We can see that the feature names in X are:

- Sepal length(cm)
- Sepal width(cm)
- Petal length(cm)
- Petal width(cm)

Note that we want to classify all the iris flowers into two categories: whether the species is Virginica or not Virginica. Moreover, we can use .astype(np.int) to convert the labels to 1 if it is Iris Virginica, else 0.

```
print(iris['feature_names'])
X = iris['data']
y = (iris['target'] == 2).astype(np.int)
```

4. Let us extract the first two data points.

```
flower_1 = X[0]
flower_2 = X[1]
```

5. Since there are four features in X, we need to invoke our euclidean function defined in Section 1. By doing that, we can calculate the distance between the two data points we selected in the previous step.

```
print(euclidean(flower_1, flower_2))
```

Section 4 – Walkthrough of the algorithm

In general, K-Nearest Neighbor consists of three steps:

- Data normalization
- Computation of k nearest neighbors
- Classification of new data points

We have already learned the z-score normalization in Lab 4, now let us explore another normalization technique named Min-Max normalization.
Without normalization, the features might have different scales. For example, there might be features that have huge values. Since the distance feature treats all dimensions equally, those features with huge scales would outweigh the importance of other features. That is why it would be a good idea if we normalize the data first, restraining them within a specific range so no feature would outweigh others to a great extent.
To implement the Min-Max normalization, we first need to find out both the maximum and the minimum value in the feature values. After that, we can scale the instance value using the following formula:

$$value(normalized) = \frac{value - minimum}{maximum - minimum}$$

1. Let us implement the Min-Max normalization in code. Write a function called `min_max_normalize()`, which takes a list as its argument.

```
import copy
```

```
def min_max_normalize(data):
    data_normalized = copy.deepcopy(data)
    for i in range(len(data[0])):
        minimum = min(data[:,i])
        maximum = max(data[:,i])
        feature_normalized = []

        for value in data[:,i]:
            value = (value - minimum) / (maximum - minimum)
            feature_normalized.append(value)
        data_normalized[:,i] = feature_normalized
    return data_normalized
```

2. Access the sepal length of the `flower_1` instance after Min-Max normalization.

```
X_normalized = min_max_normalize(X)
print(X_normalized[:,0][0])
```

3. After the data normalization step, we ought to compute the distance between the unknown instance (the point we are trying to classify) and every other known point in the dataset. We also need to define the number of neighbors to classify the unknown instance after all the distances are calculated.

```
def classify(unknown, X, k):
    distances = []

    for i in range(len(X)):
        distance_to_point = distance(X[i], unknown)
        distances.append([distance_to_point, i])
    # .sort() would sort according to the first item of the innerlists
    distances.sort()
    nearest_neighbors = distances[0:k]
    return nearest_neighbors
```

4. For now, we would set the number of neighbors(k) to 3. Let us call the classify function to return three neighbors, which are closest to an unknown point (0.2, 0.6, 0.1, 0.05). ‘Unknown’ here refers to the species.

```
print(classify([.2, .6, .1, .05], X_normalized, 3))
```

5. After finding the three closest neighbors of the point, we shall check which species these neighbors are classified. If more of the neighbors are Iris Virginica, then the unknown point would be classified as Iris Virginica as well.

Let us modify the classify() function defined in the previous step. This time we can pass the labels(y) as an additional argument.

```
def classify(unknown, X, y, k):
    distances = []
    virginica = 0
    non_virginica = 0

    for i in range(len(X)):
        distance_to_point = distance(X[i], unknown)
        distances.append([distance_to_point, i])

    distances.sort()
    nearest_neighbors = distances[0:k]

    for iris in nearest_neighbors:
        index = iris[1]
        if(y[index] == 1):
            virginica += 1
        elif(y[index] == 0):
            non_virginica += 1

    if (virginica >= non_virginica):
        result = 1
    else:
        result = 0

    return result
```

6. You can check whether the unknown point belongs to Virginica or not.

```
print(classify([.2, .6, .1, .05], X_normalized, y, 3))
```

Now you have defined a K-Nearest-Neighbors classifier. Let us explore how to implement Sci-kit Learn's KNN module in the next section.

Section 4 (cont'd) – Sci-kit Learn Implementation of K-Nearest-Neighbor

7. To evaluate the performance of the KNN classifier on the dataset, we shall first do a train_test_split. Set the percentage of the test set to 20%. X_train is the training set, X_test is the testing set, y_train contains the training labels(ground truth), and y_test

refers to the testing labels. Note that the random_state parameter assures that the data would be split in the same way every time you run the code.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_normalized, y, test_size=0.2,
random_state=2020)
```

8. Import KneighborsClassifier from sklearn.neighbors. Create an instance of KneighborsClassifier object, name it knn_model. Use .fit() method to train this model with X_train and y_train.

```
from sklearn.neighbors import KneighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors = 3)
knn_model.fit(X_train, y_train)
```

9. Print the score of your model on the testing set. The .score() method calculates the difference between the model's predicted values and the true labels.
   Print this score.

```
print(knn_model.score(X_test, y_test))
```

10. Since we can select different values of k, let us write a loop to determine which values of k would lead to the highest score on the testing set. The range of k is from 1 to the number of instances in the training set. When the number of k is very small, the model is more likely to be overfitting. After our train_test_split, we have 120 instances in the training set.
    Determine the best value of k by coding it yourself for this part (Hint: k is possible to be an even number).
    Print the best value of k.

11. Print the new score, which should be the highest score, when applying the best value of k which is obtained in Step 10.

Submit the code for Steps 10 and 11 in Section 4 with either '.ipynb' format or '.txt' format to A2L.

END